

BedLog Raspberry Pi GitHub Auto-Update Guide

Automatic deployment from MacBook / VS Code to Raspberry Pi using GitHub and systemd

Purpose: This guide documents the working BedLog update setup. A change is developed on the MacBook, pushed to GitHub, and automatically installed by the Raspberry Pi.

MacBook / VS Code	GitHub main	Raspberry Pi	BedLog Services
Edit + git push	New commit	Check every 5 min	Restart + health check

1. Confirm the Existing BedLog System Is Healthy

Before enabling automatic updates, verify the frontend and backend are running.

```
sleep 10

curl -I http://localhost:5173
curl http://localhost:4002/health
```

Expected results: the frontend returns **HTTP/1.1 200 OK** and the backend returns **{"ok":true}**.

2. Create the Automatic Update Script

On the Raspberry Pi, create the updater:

```
sudo nano /usr/local/bin/bedlog-update.sh
```

Paste the following script:

```
#!/bin/bash

set -u

APP_DIR="/home/pi/Documents/bedlog"
BRANCH="main"
LOG_TAG="bedlog-update"

log() {
    logger -t "$LOG_TAG" "$1"
    echo "$(date '+%Y-%m-%d %H:%M:%S') $1"
}

cd "$APP_DIR" || {
    log "ERROR: Cannot enter $APP_DIR"
    exit 1
}

log "Checking GitHub for BedLog updates..."

if ! git fetch origin "$BRANCH"; then
    log "ERROR: git fetch failed."
    exit 1
fi
```

```

LOCAL=$(git rev-parse HEAD)
REMOTE=$(git rev-parse "origin/$BRANCH")

if [ "$LOCAL" = "$REMOTE" ]; then
    log "BedLog is already up to date."
    exit 0
fi

log "New BedLog version detected."
log "Current: $LOCAL"
log "New:      $REMOTE"

OLD_COMMIT="$LOCAL"

log "Stopping BedLog kiosk and frontend..."
systemctl stop bedlog-kiosk.service
systemctl stop bedlog-vite.service

log "Updating BedLog source..."

if ! git reset --hard "origin/$BRANCH"; then
    log "ERROR: Git update failed."
    systemctl start bedlog-vite.service
    systemctl start bedlog-kiosk.service
    exit 1
fi

log "Installing dependencies..."

if ! npm install; then
    log "ERROR: npm install failed. Rolling back."
    git reset --hard "$OLD_COMMIT"
    systemctl restart bedlog-backend.service
    systemctl restart bedlog-vite.service
    sleep 10
    systemctl restart bedlog-kiosk.service
    exit 1
fi

log "Building BedLog..."

if ! npm run build; then
    log "ERROR: Build failed. Rolling back."
    git reset --hard "$OLD_COMMIT"
    systemctl restart bedlog-backend.service
    systemctl restart bedlog-vite.service
    sleep 10
    systemctl restart bedlog-kiosk.service
    exit 1
fi

log "Restarting BedLog services..."

systemctl restart bedlog-backend.service
systemctl restart bedlog-vite.service
sleep 10

log "Checking frontend..."

if ! curl -fsS --max-time 10 http://localhost:5173/ >/dev/null; then
    log "ERROR: Frontend health check failed. Rolling back."
    git reset --hard "$OLD_COMMIT"
    systemctl restart bedlog-backend.service
    systemctl restart bedlog-vite.service
    sleep 10
    systemctl restart bedlog-kiosk.service
    exit 1
fi

log "Checking backend..."

```

```

if ! curl -fsS --max-time 10 http://localhost:4002/health >/dev/null; then
    log "ERROR: Backend health check failed. Rolling back."
    git reset --hard "$OLD_COMMIT"
    systemctl restart bedlog-backend.service
    systemctl restart bedlog-vite.service
    sleep 10
    systemctl restart bedlog-kiosk.service
    exit 1
fi

log "Health checks passed."
systemctl restart bedlog-kiosk.service
log "BedLog successfully updated to $REMOTE."
exit 0

```

Save with Ctrl+O, Enter, Ctrl+X, then make the script executable:

```
sudo chmod +x /usr/local/bin/bedlog-update.sh
```

3. Create the systemd Update Service

```
sudo nano /etc/systemd/system/bedlog-update.service
```

```

[Unit]
Description=BedLog GitHub Auto Updater
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
User=root
WorkingDirectory=/home/pi/Documents/bedlog
ExecStart=/usr/local/bin/bedlog-update.sh

```

4. Create the 5-Minute Timer

```
sudo nano /etc/systemd/system/bedlog-update.timer
```

```

[Unit]
Description=Check GitHub for BedLog updates every 5 minutes

[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
Persistent=true

[Install]
WantedBy=timers.target

```

Reload systemd:

```
sudo systemctl daemon-reload
```

5. Fix Git Safe-Directory Permission

Because the update service runs as root while the repository belongs to the pi user, Git may report **detected dubious ownership in repository**. Configure the BedLog repository as safe for root:

```

sudo git config --global --add safe.directory /home/pi/Documents/bedlog

sudo git config --global --get-all safe.directory

```

The output should include /home/pi/Documents/bedlog.

6. Test the Updater Manually

```
sudo /usr/local/bin/bedlog-update.sh
```

When GitHub has no newer commit, the script should report that BedLog is already up to date.

7. Enable Automatic Updates

```
sudo systemctl enable --now bedlog-update.timer

systemctl status bedlog-update.timer --no-pager
systemctl list-timers bedlog-update.timer --no-pager
```

The timer should show **active (waiting)** and a next trigger approximately five minutes later.

8. Perform a Real End-to-End Test

On the MacBook, make a small visible frontend change and push it:

```
cd ~/Desktop/bedlog

git status
git add .
git commit -m "Test automatic Raspberry Pi update"
git push
```

Do not manually pull the update on the Pi. Wait for the timer. The Pi should detect the new commit, install dependencies, build the project, restart the services, and perform health checks.

Check the installed commit:

```
cd /home/pi/Documents/bedlog
git log -1 --oneline
```

Inspect the updater log:

```
sudo journalctl -u bedlog-update.service -n 100 --no-pager
```

9. Verify BedLog After an Update

```
systemctl is-active bedlog-backend
systemctl is-active bedlog-vite
systemctl is-active bedlog-kiosk

curl -I http://localhost:5173
curl http://localhost:4002/health
```

Expected: all three services are active, frontend returns HTTP 200, and backend returns `{"ok":true}`.

10. Confirmed Successful Test

The working test updated the Pi from commit **89f9e0a** to **e9e399a - Test automatic Raspberry Pi update**. The build completed successfully and both health checks passed.

Important Notes Before Cloning the Raspberry Pi

Database protection: The updater resets the Git working tree to origin/main, but the PostgreSQL database is external to the Git repository and is not erased by this update process.

Do not blindly run `npm audit fix --force`. npm reported one high-severity dependency vulnerability during the successful test, but forcing dependency upgrades can introduce breaking changes. Review dependency updates separately.

Before cloning the master SD card: clean the duplicate `/home/pi/Documents/bedlog/bedlog` directory and `__MACOSX` metadata after confirming they are unnecessary; protect machine-specific device identity/configuration; and consider adding a randomized timer delay so many deployed Pis do not update simultaneously.

Normal Development Workflow

After this setup, normal BedLog updates are simple: edit the project on the MacBook in VS Code, test it, commit the changes, and push to GitHub. The Raspberry Pi checks automatically every five minutes and only performs the deployment when origin/main contains a different commit.